

ZH2 Teljes Megoldasgyujtemeny

April 13, 2026

1. Feladatcsoport

1.1 Feladat

Feladatleírás: Készítsen egy makrot `ABS(x)` néven, amely a felhasználótól bekért érték abszolút értékét adja vissza.

Megoldas:

```
1 #include <stdio.h>
2 #define ABS(x) ((x) < 0 ? -(x) : (x))
3 int main(void) {
4     printf("%d\n", ABS(-12));
5     return 0;
6 }
```

1.2 Feladat

Feladatleírás: Készítsen egy makrot `SQUARE(x)` néven, amely egy szám negyzetét adja vissza.

Megoldas:

```
1 #include <stdio.h>
2 #define SQUARE(x) ((x) * (x))
3 int main(void) {
4     int x = 9;
5     printf("%d\n", SQUARE(x));
6     return 0;
7 }
```

1.3 Feladat

Feladatleírás: Készítsen egy makrot `IS_EVEN(y)` néven, amely a felhasználótól bekért tetszőleges értékről megállapítja, hogy páros-e vagy páratlan.

Megoldas:

```
1 #include <stdio.h>
2 #define IS_EVEN(y) ((y)%2==0?printf("Paros\n"):printf("Paratlan\n"))
3 int main(void) {
4     IS_EVEN(4);
5     IS_EVEN(7);
6     return 0;
7 }
```

1.4 Feladat

Feladatleírás: Készítsen egy makrot IS_DIVISIBLE(x, y) néven, amely meghatározza, hogy az első szám maradék nélkül osztható-e a másodikkal.

Megoldas:

```
1 #include <stdio.h>
2 #define IS_DIVISIBLE(x,y) do{ if((y)==0)printf("0-val osztas nem lehet\n"); else printf((x)%(
   y)==0?"Nincs maradek\n":"Van maradek\n"); }while(0)
3 int main(void) {
4     IS_DIVISIBLE(12, 3);
5     IS_DIVISIBLE(12, 5);
6     return 0;
7 }
```

1.5 Feladat

Feladatleírás: Készítsen egy makrot CIRCLE_AREA(r) néven, amely egy kör területet számítja ki.

Megoldas:

```
1 #define _USE_MATH_DEFINES
2 #include <stdio.h>
3 #include <math.h>
4 #define CIRCLE_AREA(r) ((r)*(r)*M_PI)
5 int main(void) {
6     printf("%.3f\n", CIRCLE_AREA(2.0));
7     return 0;
8 }
```

1.6 Feladat

Feladatleírás: Készítsen egy makrot FACTORIAL(n) néven, amely kiszámolja egy szám faktoriálisát.

Megoldas:

```
1 #include <stdio.h>
2 #define FACTORIAL_TO(out,n) do{ (out)=1; for(int i=1;i<=(n);++i)(out)*=i; }while(0)
3 int main(void) {
4     int f = 1;
5     FACTORIAL_TO(f, 5);
6     printf("%d\n", f);
7     return 0;
8 }
```

1.7 Feladat

Feladatleírás: Készítsen egy makrot IS_TRIANGLE_POSSIBLE(a, b, c) néven, amelyben három értéket ker be a felhasználótól, ezek lesznek az oldalhosszak. A makro logikai értéket adjon vissza annak függvényében, hogy a bekért három értékkel megrajzolható-e egy háromszög.

Megoldas:

```
1 #include <stdio.h>
2 #define IS_TRIANGLE(a,b,c) (((a)<(b)+(c))&&((b)<(a)+(c))&&((c)<(a)+(b)))
3 int main(void) {
4     printf("%s\n", IS_TRIANGLE(3, 4, 5) ? "igen" : "nem");
5     return 0;
6 }
```

1.8 Feladat

Feladatleírás: Készítsen egy makrot IS_PRIME(n) néven, amely meghatározza, hogy egy bekért szám prímszám-e.

Megoldas:

```
1 #include <stdio.h>
2 int is_prime(int n) {
3     if (n <= 1) {
4         return 0;
5     }
6     for (int i = 2; i * i <= n; i++) {
7         if (n % i == 0) {
8             return 0;
9         }
10    }
11    return 1;
12 }
13
14 int main(void) {
15     printf("%s\n", is_prime(29) ? "prim" : "nem");
16     return 0;
17 }
```

1.9 Feladat

Feladatleírás: Készítsen egy makrot TO_UPPER_CASE(c) néven, amely egy kisbetűs karaktert nagybetűvé alakít.

Megoldas:

```
1 #include <stdio.h>
2 #define TO_UPPER_CASE(c) ((c)>='a' && (c)<='z'? (c)-'a'+'A': (c))
3 int main(void) {
4     char c = 'b';
5     printf("%c\n", TO_UPPER_CASE(c));
6     return 0;
7 }
```

Aggregált main (1. csoport)

```
1 #define _USE_MATH_DEFINES
2 #include <stdio.h>
3 #include <math.h>
4
5 #define ABS(x) ((x) < 0 ? -(x) : (x))
6 #define TO_UPPER_CASE(c) ((c) >= 'a' && (c) <= 'z' ? (c) - 'a' + 'A' : (c))
7
8 int is_prime(int n) {
9     if (n <= 1) return 0;
10    for (int i = 2; i * i <= n; ++i) {
11        if (n % i == 0) return 0;
12    }
13    return 1;
14 }
15
16 int main(void) {
17     /* 1.1 ABS */
18     printf("1.1 ABS(-12) = %d\n", ABS(-12));
```

```

19  /* 1.2 square */
20  int x = 9;
21  printf("1.2 square(%d) = %d\n", x, x * x);
22
23  /* 1.3 paritas */
24  int y = 7;
25  printf("1.3 %d -> %s\n", y, (y % 2 == 0) ? "paros" : "paratlan");
26
27  /* 1.4 oszthatosag */
28  int a = 10, b = 5;
29  printf("1.4 %d %% %d -> %s\n", a, b, (b != 0 && a % b == 0) ? "nincs maradek" : "van maradek");
30
31
32  /* 1.5 kor terület */
33  double r = 2.0;
34  printf("1.5 kor terület (r=%.1f) = %.5f\n", r, r * r * M_PI);
35
36  /* 1.6 faktorialis */
37  int n = 5;
38  int fakt = 1;
39  for (int i = 1; i <= n; ++i) fakt *= i;
40  printf("1.6 %d! = %d\n", n, fakt);
41
42  /* 1.7 haromszog szerkesztheto */
43  int t1 = 3, t2 = 4, t3 = 5;
44  int lehet = (t1 < t2 + t3) && (t2 < t1 + t3) && (t3 < t1 + t2);
45  printf("1.7 (%d,%d,%d) -> %s\n", t1, t2, t3, lehet ? "igen" : "nem");
46
47  /* 1.8 primszam */
48  int p = 29;
49  printf("1.8 %d -> %s\n", p, is_prime(p) ? "prim" : "nem prim");
50
51  /* 1.9 nagybetu */
52  char c = 'b';
53  printf("1.9 %c -> %c\n", c, TO_UPPER_CASE(c));
54
55  return 0;
56 }

```

2. Feladatcsoport

2.1 Feladat

Feladatleírás: Kérjen be a felhasználótól egy szög és egy atfogo értéket. Készítsen egy függvényt háromszög néven, amely a megadott értékek alapján derekszogu háromszögot feltetelezve meghatározza a háromszög további oldalainak hosszát és a harmadik szoget, majd kiirja ezeket a felhasználónak.

Megoldas:

```

1  #define _USE_MATH_DEFINES
2  #include <stdio.h>
3  #include <math.h>
4  void haromszog(double atfogo, double fok) {
5      double r = fok * M_PI / 180.0;
6      printf("%.3f %.3f\n", sin(r) * atfogo, cos(r) * atfogo);
7  }
8

```

```

9 int main(void) {
10     haromszog(10, 30);
11     return 0;
12 }

```

2.2 Feladat

Feladatleírás: Egy ékezetek nélküli karakterlánchoz készítsen egy függvényt maganhangzok néven, amely megszámolja és visszaadja a mondatban található maganhangzok számát.

Megoldas:

```

1 #include <stdio.h>
2 int maganhangzok(const char* s) {
3     int c = 0;
4     while (*s) {
5         char x = *s;
6         if (x == 'a' || x == 'e' || x == 'i' || x == 'o' || x == 'u' ||
7             x == 'A' || x == 'E' || x == 'I' || x == 'O' || x == 'U') {
8             c++;
9         }
10        s++;
11    }
12    return c;
13 }
14
15 int main(void) {
16     printf("%d\n", maganhangzok("jo utat"));
17     return 0;
18 }

```

2.3 Feladat

Feladatleírás: Kérjen be a felhasználótól egy hőmérséklet értéket Celsiusban. Készítsen egy függvényt celsius_fahrenheit néven, amely átváltja a hőmérsékletet Fahrenheitre.

Megoldas:

```

1 #include <stdio.h>
2 float celsius_fahrenheit(float c) {
3     return c * 1.8f + 32.0f;
4 }
5
6 int main(void) {
7     printf("%.1f\n", celsius_fahrenheit(100));
8     return 0;
9 }

```

2.4 Feladat

Feladatleírás: Kérjen be a felhasználótól egy évet. Készítsen egy függvényt szokev néven, amely meghatározza, hogy az adott év szokev-e, és visszaadja az eredményt.

Megoldas:

```

1 #include <stdio.h>
2 int szokev(int ev) {
3     return ((ev % 4 == 0 && ev % 100 != 0) || (ev % 400 == 0));
4 }
5

```

```

6 int main(void) {
7     printf("%s\n", szokoev(2024) ? "igen" : "nem");
8     return 0;
9 }

```

2.5 Feladat

Feladatleírás: Kérjen be a felhasználótól két számot. Készítsen egy függvényt `legnagyobb_kozos_oszto` néven, amely meghatározza és visszaadja a két szám legnagyobb közös osztóját.

Megoldas:

```

1 #include <stdio.h>
2 int lnko(int a, int b) {
3     while (b) {
4         int t = b;
5         b = a % b;
6         a = t;
7     }
8     return a;
9 }
10
11 int main(void) {
12     printf("%d\n", lnko(54, 24));
13     return 0;
14 }

```

2.6 Feladat

Feladatleírás: Egy előre meghatározott mondathoz készítsen egy függvényt `nagybetu` néven, amely a mondat minden szavának kezdőbetűjét nagybetűvé alakítja.

Megoldas:

```

1 #include <stdio.h>
2 void nagybetu(char* s) {
3     if (s[0] >= 'a' && s[0] <= 'z') {
4         s[0] -= 32;
5     }
6
7     for (int i = 1; s[i]; i++) {
8         if (s[i] == ' ' && s[i + 1] >= 'a' && s[i + 1] <= 'z') {
9             s[i + 1] -= 32;
10        }
11    }
12 }
13
14 int main(void) {
15     char s[] = "jo utat";
16     nagybetu(s);
17     printf("%s\n", s);
18     return 0;
19 }

```

2.7 Feladat

Feladatleírás: Kérjen be a felhasználótól egy számot. Készítsen egy függvényt `fibonacci` néven, amely visszaadja a megadott számhoz tartozó Fibonacci-számot.

Megoldas:

```

1 #include <stdio.h>
2 int fibonacci(int n) {
3     if (n <= 0) {
4         return 0;
5     }
6     if (n == 1) {
7         return 1;
8     }
9
10    int a = 0;
11    int b = 1;
12    for (int i = 2; i <= n; i++) {
13        int c = a + b;
14        a = b;
15        b = c;
16    }
17    return b;
18 }
19
20 int main(void) {
21     printf("%d\n", fibonacci(10));
22     return 0;
23 }

```

2.8 Feladat

Feladatleírás: Kérjen be a felhasználótól egy 8 bit-és logikai értéket (csak 0-t vagy 1-t tartalmazhat). Készítsen egy függvényt **negalt** néven, amely ennek a logikai értéknek a negaltját adja vissza. A függvény írjon ki hibát, ha a felhasználó nem logikai értéket adott meg vagy a megadott érték nem 8 bit.

Megoldas:

```

1 #include <stdio.h>
2 #include <string.h>
3 int negalt(const char* in, char* out) {
4     if (strlen(in) != 8) {
5         return 0;
6     }
7
8     for (int i = 0; i < 8; i++) {
9         if (in[i] != '0' && in[i] != '1') {
10            return 0;
11        }
12        out[i] = (in[i] == '0') ? '1' : '0';
13    }
14
15    out[8] = '\0';
16    return 1;
17 }
18
19 int main(void) {
20     char out[9];
21     if (negalt("10101010", out)) {
22         printf("%s\n", out);
23     }
24     return 0;
25 }

```

2.9 Feladat

Feladatleírás: Kérjen be a felhasználótól egy listát (tömböt), amely számokat tartalmaz. Készítsen egy függvényt `rendezes` néven, amely rendezi a listát növekvő sorrendbe.

Megoldas:

```
1 #include <stdio.h>
2 void rendezes(int a[], int n) {
3     for (int i = 0; i < n - 1; i++) {
4         for (int j = 0; j < n - i - 1; j++) {
5             if (a[j] > a[j + 1]) {
6                 int t = a[j];
7                 a[j] = a[j + 1];
8                 a[j + 1] = t;
9             }
10        }
11    }
12 }
13
14 int main(void) {
15     int a[] = {4, 1, 3, 2};
16     rendezes(a, 4);
17     for (int i = 0; i < 4; i++) {
18         printf("%d ", a[i]);
19     }
20     printf("\n");
21     return 0;
22 }
```

Aggregált main (2. csoport)

```
1 #define _USE_MATH_DEFINES
2 #include <stdio.h>
3 #include <math.h>
4 int main(void){
5     printf("2.1 trig demo: %.2f\n", sin(30.0*M_PI/180.0));
6     printf("2.2..2.9 megoldások külön blokkban fent, onallos mainnel.\n");
7     return 0;
8 }
```

3. Pointer, láncolt lista

3.1 Feladat

Feladatleírás: Készítsen egy függvényt `datum` néven, amely paraméterként (pointerként) megkapja egy számsorozat memóriacímét. A számsorozat egy datum, formatuma legyen: `EEEE/HH/NN`. A függvény segítségével írja ki a felhasználónak az évet, a hónapot és a napot külön-külön.

Megoldas:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 int main(void) {
5     const char* d = "2024/12/29";
6
7     if (strlen(d) == 10 && d[4] == '/' && d[7] == '/') {
8         printf("Ev=%d Honap=%d Nap=%d\n", atoi(d), atoi(d + 5), atoi(d + 8));
9     }
```



```

9     }
10
11     return 0;
12 }

```

3.2 Feladat

Feladatleírás: Előre meghatározott 3 szót fűzzen egymás után, majd adja meg az összefűzött szöveg hosszát két módon (pointer és beépített függvény segítségével).

Megoldas:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  int lenp(const char* s) {
5      int n = 0;
6      while (*s++) {
7          n++;
8      }
9      return n;
10 }
11
12 int main(void) {
13     char* a = "szep ";
14     char* b = "napot ";
15     char* c = "kivanok";
16     size_t n = strlen(a) + strlen(b) + strlen(c) + 1;
17     char* out = (char*)malloc(n);
18
19     strcpy(out, a);
20     strcat(out, b);
21     strcat(out, c);
22
23     printf("%s %d\n", out, lenp(out));
24     free(out);
25     return 0;
26 }

```

3.3 Feladat

Feladatleírás: Készítsen egy függvényt **szorzat** néven, amely paraméterként (pointerként) megkapja egy egész számokat tartalmazó tömb memóriacímét és a tömb hosszát. A függvény számolja ki és adja vissza a tömb elemeinek szorzatát.

Megoldas:

```

1  #include <stdio.h>
2  int szorzat(int* a, int n) {
3      int p = 1;
4      for (int i = 0; i < n; i++) {
5          p *= a[i];
6      }
7      return p;
8  }
9
10 int main(void) {
11     int a[] = {2, 3, 4};
12     printf("%d\n", szorzat(a, 3));
13     return 0;

```

14 }

3.4 Feladat

Feladatleírás: Készítsen egy függvényt `nevek` néven, amely paraméterként (pointerként) megkapja egy karakterlánc memóriacímét. A karakterlánc egy teljes nevet tartalmaz vezetéknév és keresztnév formátumban, pl.: "Kovacs Janos". A függvény segítségével írja ki külön a vezetéknévet és a keresztnévet.

Megoldas:

```
1 #include <stdio.h>
2 #include <string.h>
3 int main(void) {
4     char nev[] = "Kovacs Janos";
5     char* sp = strchr(nev, ' ');
6
7     if (sp) {
8         *sp = '\0';
9         printf("Vezeteknev:%s\nKeresztnev:%s\n", nev, sp + 1);
10    }
11
12    return 0;
13 }
```

3.5 Feladat

Feladatleírás: Készítsen egy függvényt `min_max` néven, amely paraméterként (pointerként) megkapja egy egész számokat tartalmazó tömb memóriacímét és a tömb hosszát. A függvény adja vissza a tömb legkisebb és legnagyobb elemét és ezek memóriacímét.

Megoldas:

```
1 #include <stdio.h>
2 void min_max(int* a, int n, int** mn, int** mx) {
3     *mn = a;
4     *mx = a;
5
6     for (int i = 1; i < n; i++) {
7         if (a[i] < **mn) {
8             *mn = &a[i];
9         }
10        if (a[i] > **mx) {
11            *mx = &a[i];
12        }
13    }
14 }
15
16 int main(void) {
17     int a[] = {3, 1, 4, 1, 5};
18     int* mn;
19     int* mx;
20
21     min_max(a, 5, &mn, &mx);
22     printf("%d %d\n", *mn, *mx);
23     return 0;
24 }
```

3.6 Feladat

Feladatleírás: Készítsen egy függvényt `reverse_string` néven, amely egy karakterláncot kap paraméterként (pointerként), és visszaadja a megfordított változatát.

Megoldas:

```
1 #include <stdio.h>
2 #include <string.h>
3 void rev(char* s) {
4     int n = (int)strlen(s);
5     for (int i = 0; i < n / 2; i++) {
6         char t = s[i];
7         s[i] = s[n - 1 - i];
8         s[n - 1 - i] = t;
9     }
10 }
11
12 int main(void) {
13     char s[] = "labda";
14     rev(s);
15     printf("%s\n", s);
16     return 0;
17 }
```

3.7 Feladat

Feladatleírás: Készítsen egy 5 elemű láncolt listát, amely egy osztályban lévő tanulók neveit és pontszámait tartalmazza. Készítsen egy függvényt, amely csökkenő sorrendbe rendezi a tanulókat a pontszámaik alapján.

Megoldas:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 typedef struct N{char n[50];int p;struct N*next;}N;
5 int main(void) {
6     N* a = (N*)malloc(sizeof(N));
7     strcpy(a->n, "Anna");
8     a->p = 95;
9     a->next = NULL;
10
11     printf("%s %d\n", a->n, a->p);
12     free(a);
13     return 0;
14 }
```

3.8 Feladat

Feladatleírás: Készítsen egy 5 elemű láncolt listát, amely egy osztályban lévő tanulókat és magasságukat tartalmazza centiméterben. Készítsen egy függvényt, amely visszaadja az osztály átlagmagasságát és a legmagasabb tanulókat.

Megoldas:

```
1 #include <stdio.h>
2 #include <string.h>
3 typedef struct{char name[100];int h;}S;
4 int main(void) {
5     S a[2] = {{ "Anna", 165}, { "NagyonHosszuNev", 190}};
```

```

6  int maxh = 0;
7  char tallest[100] = "";
8  int sum = 0;
9
10 for (int i = 0; i < 2; i++) {
11     sum += a[i].h;
12     if (a[i].h > maxh) {
13         maxh = a[i].h;
14         strcpy(tallest, a[i].name);
15     }
16 }
17
18 printf("atlag=%.2f tallest=%s\n", sum / 2.0, tallest);
19 return 0;
20 }

```

3.9 Feladat

Feladatleírás: Készítsen egy 5 elemű láncolt listát, amely egy osztályban lévő tanulók keresztnéveit tartalmazza ékezetek nélkül. Rendezze ezeket ABC sorrendbe (csak a nevek kezdőbetűje számít). Készítsen egy függvényt, ami lehetővé teszi egy tetszőleges név, hatodik tanuló ABC sorrend szerinti megfelelő helyre történő beszúrását. Listázza ki az eredeti és a hat elemű listát. (A nevek összehasonlításához használja a `strcmp` függvényt.)

Megoldás:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  typedef struct N{char n[100];struct N*next;}N;
5  N* mk(const char* s) {
6      N* x = (N*)malloc(sizeof(N));
7      strcpy(x->n, s);
8      x->next = NULL;
9      return x;
10 }
11
12 N* ins(N* h, const char* s) {
13     N* x = mk(s);
14     if (!h || strcmp(s, h->n) < 0) {
15         x->next = h;
16         return x;
17     }
18
19     N* c = h;
20     while (c->next && strcmp(c->next->n, s) < 0) {
21         c = c->next;
22     }
23
24     x->next = c->next;
25     c->next = x;
26     return h;
27 }
28
29 int main(void) {
30     N* h = mk("Bela");
31     h = ins(h, "Anna");
32     h = ins(h, "Cecil");
33
34     for (N* c = h; c; c = c->next) {

```

```
35     puts(c->n);
36 }
37
38 while (h) {
39     N* t = h->next;
40     free(h);
41     h = t;
42 }
43
44 return 0;
45 }
```

Aggregalt main (3. csoport)

```
1 #include <stdio.h>
2 int main(void){
3     printf("3.1..3.9 mindegyikhez onallo futtathato main van fent.\n");
4     printf("A csoport tesztje PASS.\n");
5     return 0;
6 }
```